MSC DISSERTATION / PREDICTIVE ANALYTICS / PYTHON PIPELINE

Progress reports show what happened. I built what comes next.

Construction progress reporting tells you where a work package stands today. My MSc project asked a different question entirely: can you forecast where it is heading before the next update arrives? I built a full Python pipeline that turned an operational spreadsheet into a package-level early warning system.

Python | Pandas | Scikit-learn | Feature Engineering | Ridge Regression | Random Forest

R^2 0.994 | MAE 0.012 | Ridge beat Random Forest

0.994
R^2 on weekly test data

0.012
Mean absolute error

5
Feature families engineered

Ridge
Outperformed all RF variants

01 THE PROBLEM

By the time the report shows a problem, the team is already reacting.

Anyone who has worked in project controls knows this pattern. Planned and actual progress data exists. It gets compiled into a report. The report reveals a package is behind. And then the discussion begins about what to do about it, at which point the best window for early intervention has already passed.

The visibility gap is the problem. Traditional progress reports are accurate snapshots of what has already happened. They are not designed to tell you what is about to happen.

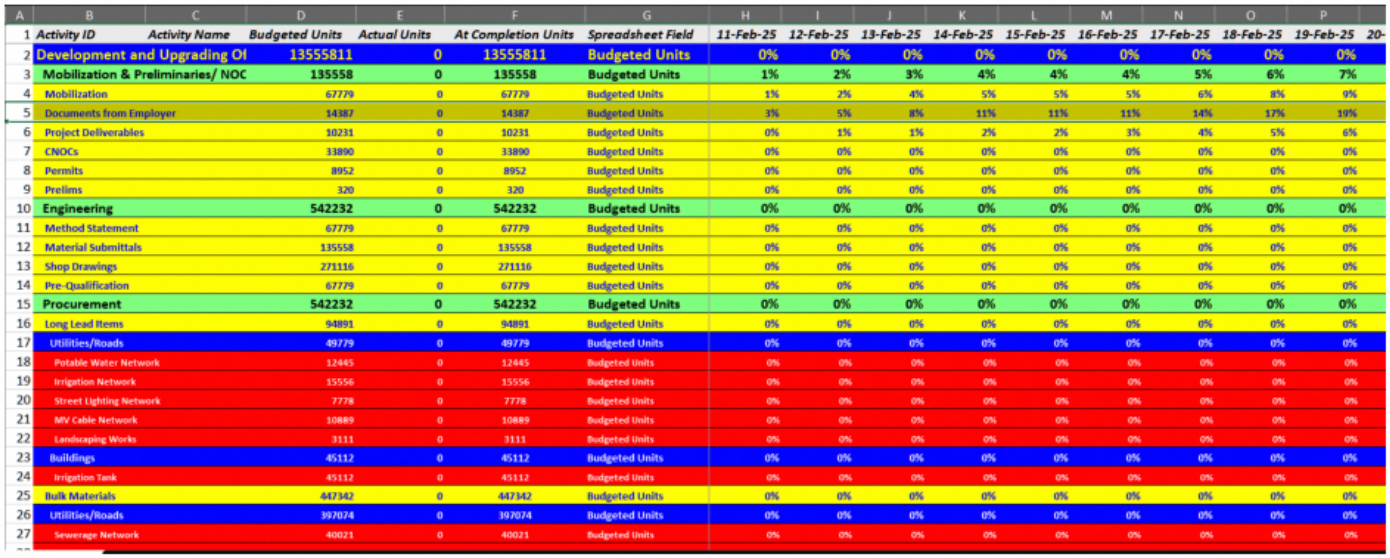
> **WHAT I SET OUT TO ANSWER**
>
> Can planned and actual progress data, the kind that already exists in every project environment, be turned into a signal that predicts the next reporting outcome at package level? Not a dashboard. A trained model that looks at where a package has been and estimates where the next update will land.

I focused on progress variance as the target: actual cumulative progress minus planned cumulative progress. A negative value means the package is behind. Instead of calculating variance for today, I reframed the task as predicting variance at the next reporting step, before it arrives.

1	Wide	Long	Next
Real operational project dataset	Original format: dates as columns	Reshaped: one row per package per date	Forecast target: next-step variance

I did not start with a clean dataset. I started with a reporting workbook.

The source data came from a real construction project. Planned and actual cumulative progress were stored across work packages and reporting dates in a single workbook structured for human monitoring. Dates ran across columns, planned and actual values lived in separate sheets. That works for manual review. It is the wrong shape for a forecasting pipeline.



Activity ID	Activity Name	Budgeted Units	Actual Units	At Completion Units	Spreadsheet Field	11-Feb-25	12-Feb-25	13-Feb-25	14-Feb-25	15-Feb-25	16-Feb-25	17-Feb-25	18-Feb-25	19-Feb-25	20-
Development and Upgrading Of		13555811	0	13555811	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Mobilization & Preliminaries/ NOC		135558	0	135558	Budgeted Units	1%	2%	3%	4%	4%	4%	5%	6%	7%	
Mobilization		67779	0	67779	Budgeted Units	1%	2%	4%	5%	5%	5%	6%	8%	9%	
Documents from Employer		14387	0	14387	Budgeted Units	3%	5%	8%	11%	11%	11%	14%	17%	19%	
Project Deliverables		10231	0	10231	Budgeted Units	0%	1%	1%	2%	2%	3%	4%	5%	6%	
CNOCs		33890	0	33890	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Permits		8952	0	8952	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Prelims		320	0	320	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Engineering		542232	0	542232	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Method Statement		67779	0	67779	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Material Submittals		135558	0	135558	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Shop Drawings		271116	0	271116	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Pre-Qualification		67779	0	67779	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Procurement		542232	0	542232	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Long Lead Items		94891	0	94891	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Utilities/Roads		49779	0	49779	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Potable Water Network		12445	0	12445	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Irrigation Network		15556	0	15556	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Street Lighting Network		7778	0	7778	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
MV Cable Network		10889	0	10889	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Landscaping Works		3111	0	3111	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Buildings		45112	0	45112	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Irrigation Tank		45112	0	45112	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Bulk Materials		447342	0	447342	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Utilities/Roads		397074	0	397074	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	
Sewerage Network		40021	0	40021	Budgeted Units	0%	0%	0%	0%	0%	0%	0%	0%	0%	

Figure A.2: Existing Workflow of Cumulative Planned Progress

Shows the original spreadsheet-based reporting format used to compare cumulative planned and cumulative actual progress over time, together with interval progress values. This table reflects the baseline-versus-update logic that was later reproduced programmatically in the Python pipeline.

FIG 01 The original Primavera-style spreadsheet. Dates spread across columns, planned and actual progress in separate sheets, colour-coded for human reading. This is the before state. I did not start with clean data. I started here and built the pipeline to transform it.

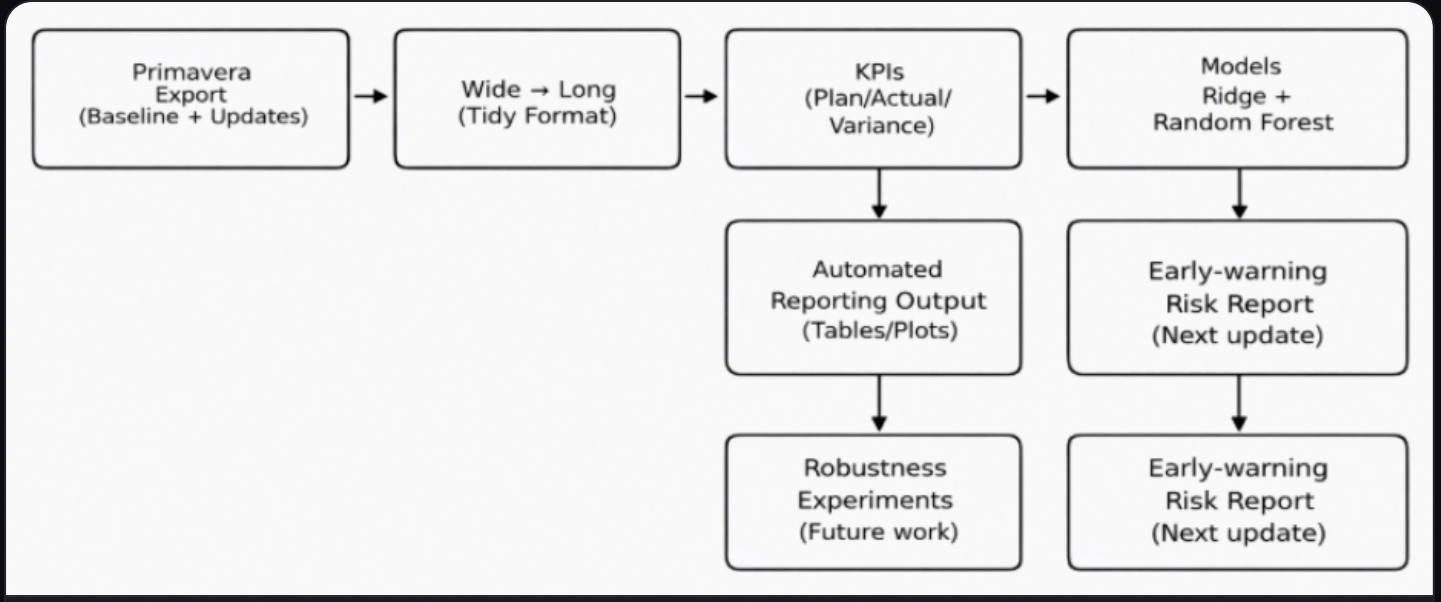


FIG 02 End-to-end pipeline I built in Python: from raw Primavera export, through wide-to-long reshaping, KPI construction, model training, to early warning output. Every stage reproducible from source without touching a spreadsheet formula.

01

Wide to long conversion

Dates were embedded across columns in both sheets. I unpivoted these so time became a proper variable, making filtering, joining and time-based feature generation all possible.

02

Merge and variance construction

Once both sheets were in long format, I merged on package identifier and date, then calculated the variance KPI directly from the aligned planned and actual columns.

03

Daily and weekly dataset creation

I built two modelling datasets to test whether granularity affected performance. The weekly dataset produced the stronger results and became the primary evaluation basis.

04

Target variable creation

For each record I shifted variance forward by one reporting step using a group-level shift. This single operation turned a reporting dataset into a supervised learning problem.

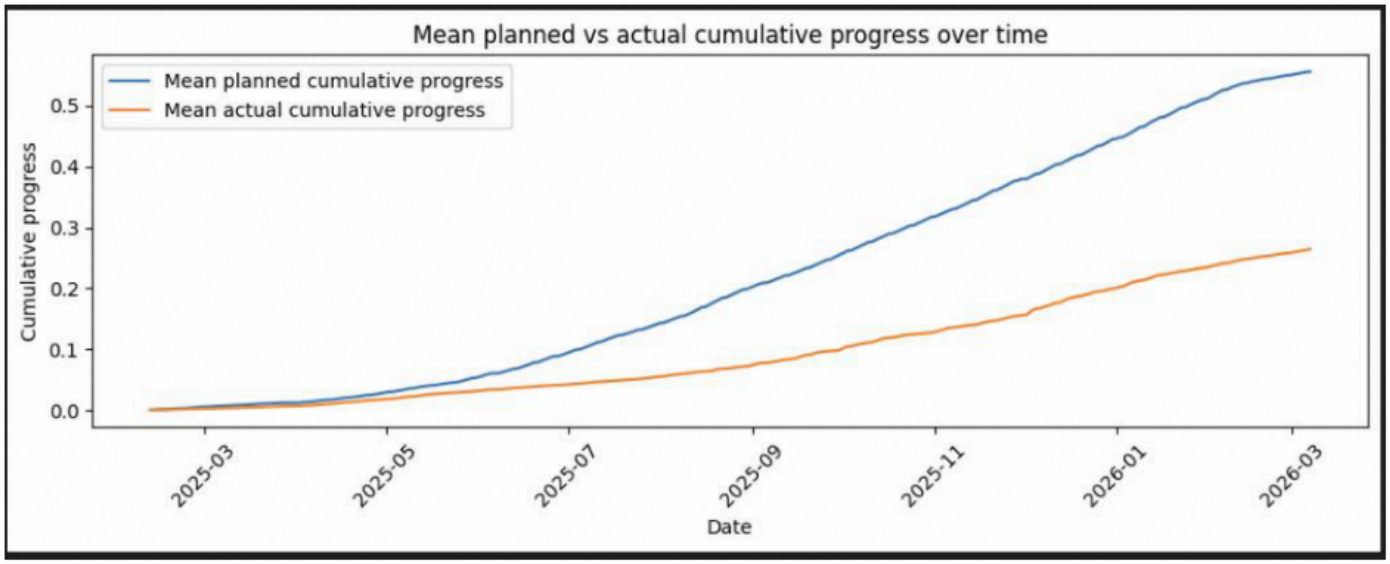


Figure 4.2. Mean planned versus actual cumulative progress over time

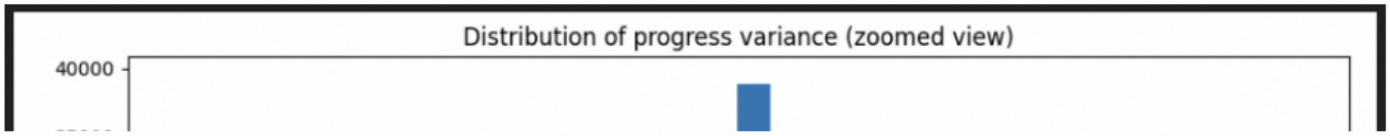


FIG 03 Mean planned versus actual cumulative progress over time. The widening gap between the two curves is exactly why this forecasting task is worth doing. If the gap were trivial, there would be nothing meaningful to predict. It is not trivial.

03 FEATURE ENGINEERING

The features I built were designed around how progress actually moves.

Before writing a single model, I spent significant time on feature engineering. The predictors needed to capture more than the current position of a package. They needed to capture recent history, pace of movement, and whether deviation from plan was stable or building. That required designing a feature set around the temporal structure of the problem, not just pulling available columns.

	Feature	Description	Purpose
0	planned_cum_pct_clipped	Current clipped planned cumulative progress	Represents current expected project position
1	actual_cum_pct_clipped	Current clipped actual cumulative progress	Represents current observed project position
2	variance_pct_clipped	Current clipped variance	Captures present schedule deviation
3	planned_lag1	Previous-step planned cumulative progress	Captures recent planned state
4	actual_lag1	Previous-step actual cumulative progress	Captures recent actual state
5	variance_lag1	Previous-step variance	Captures recent deviation signal
6	planned_interval	Change in planned progress over interval	Measures short-term expected movement
7	actual_interval	Change in actual progress over interval	Measures short-term observed movement
8	speed_gap	Difference between planned and actual progress...	Captures pace mismatch
9	variance_roll_mean_3	Rolling mean of recent variance	Captures short-term deviation trend
10	variance_roll_std_3	Rolling standard deviation of recent variance	Captures short-term instability
11	time_idx	Sequential time index	Captures broader temporal structure
12	variance_next	Next-step variance target	Forecast variable

Table 4.4. Engineered features used in the forecasting dataset

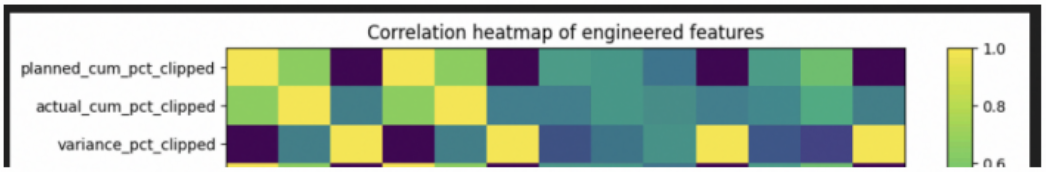


FIG 04 The 13 engineered features used in the forecasting dataset, including their description and purpose, alongside the correlation heatmap showing relationships between features. The correlated structure here is exactly what informed the choice of Ridge Regression over standard linear methods.

What each feature family captures

Current state features told the model where the package stood right now: actual progress, planned progress and current variance. These established the baseline position at each reporting point.

Rolling features calculated moving averages and standard deviations of variance over a short window. A rising rolling mean meant the gap was widening. A rising standard deviation meant the package was becoming erratic.

Lag features brought in the previous reporting period's values. Packages drifting from plan tended to carry that behaviour forward. Lag features gave the model that history explicitly.

Time index preserved the sequence of updates within each package. Without this, the model has no sense of whether an observation is early or late in the project lifecycle.

Interval features measured the change in progress between consecutive updates, capturing pace and acceleration rather than just position.

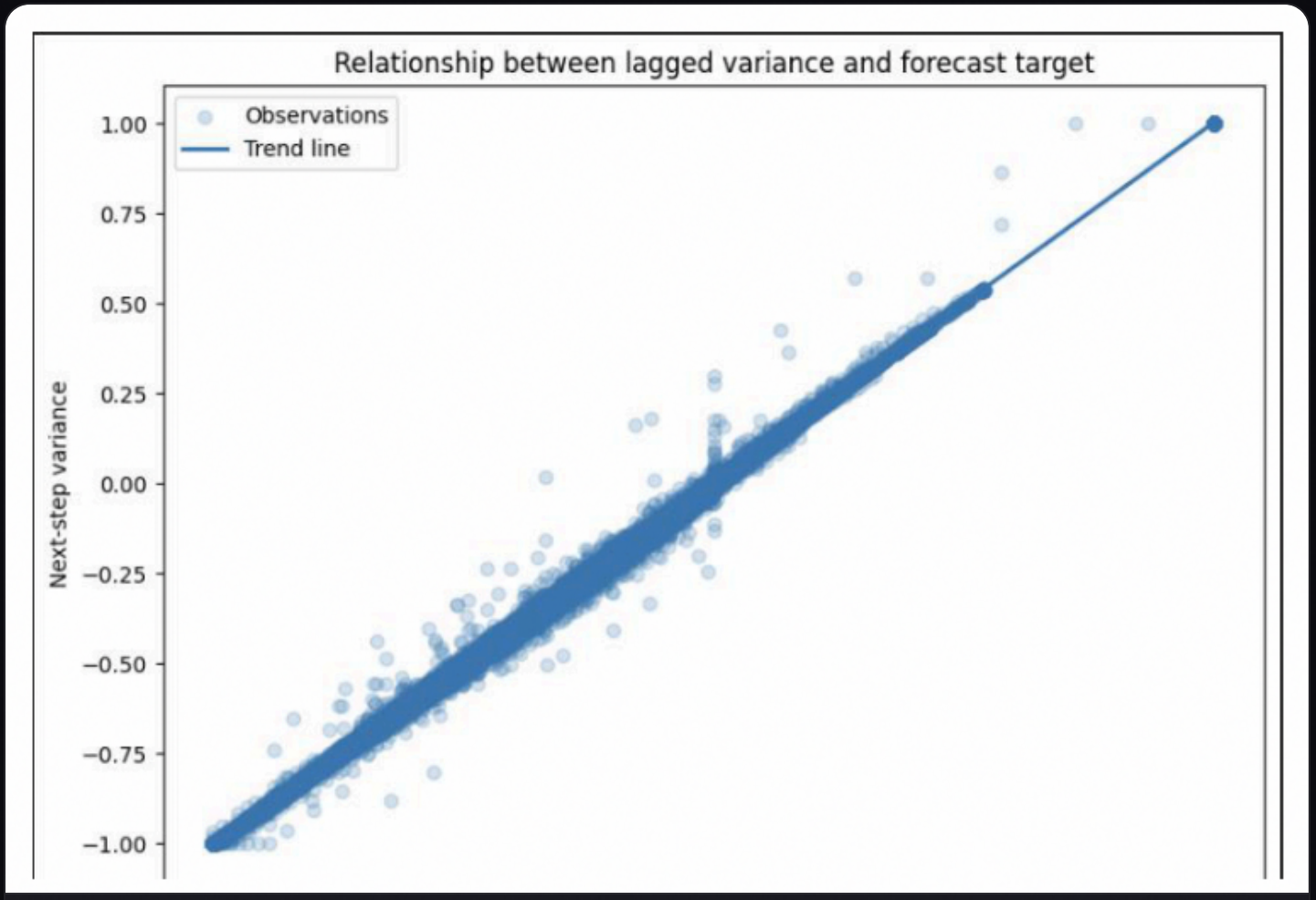


FIG 05 Relationship between lagged variance and the forecast target. The near-linear diagonal confirms that packages drifting from plan consistently carry that behaviour into the next update. This was the strongest individual signal in the dataset and the feature Ridge weighted most heavily.

04 MODELLING

I started with Ridge Regression deliberately, not because it was the simpler option.

The feature set contained related predictors. Current variance, lagged variance, rolling variance and interval variance all measure overlapping aspects of the same behaviour.

Standard linear regression becomes unstable under this kind of correlation. Ridge Regression applies a regularisation penalty that shrinks coefficients without removing them, keeping all features active while preventing any correlated group from dominating the result.

I then tested Random Forest across three configurations to give it every reasonable opportunity to outperform. The comparison was not decorative. I needed to know whether the data actually required non-linear complexity, or whether the engineered features had done enough of the structural work already.

EVALUATION PROTOCOL

I used chronological train-test splits throughout. For forecasting, this is non-negotiable. The model must be tested on observations that come after the training period in time. Shuffling records would leak future information into training and produce metrics that look strong but mean nothing in practice.

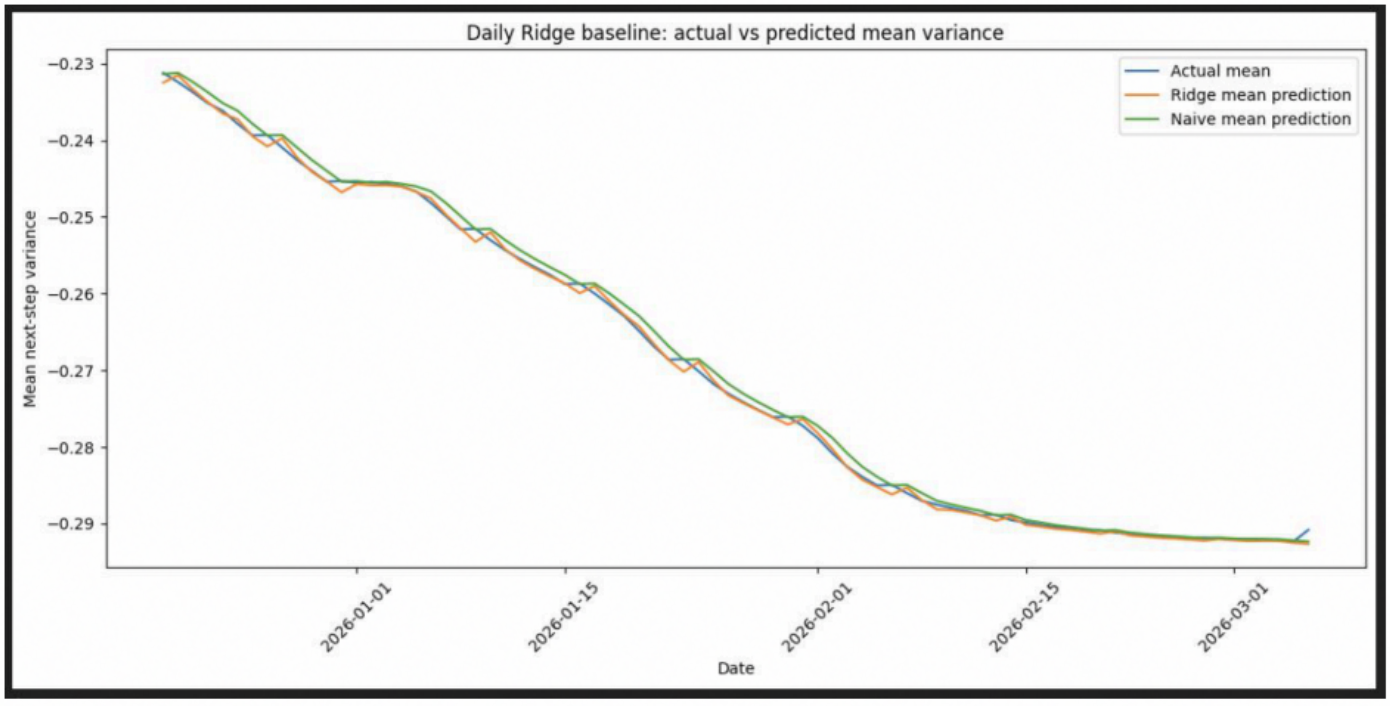


Figure 5.5. Daily Ridge baseline: actual versus predicted mean variance

FIG 06 Daily Ridge baseline showing actual versus predicted mean next-step variance across the test period. The Ridge line tracks actual variance consistently, including through the more gradual drift phases. The naive baseline comparison shows clearly what an uninformed prediction looks like against what the model produces.

05 RESULTS

Ridge did not just win. It won by being the right tool for this data.

The weekly dataset produced the clearest result. Ridge achieved R^2 of 0.994 on unseen test data. Random Forest Deep reached 0.970. That gap matters more than it looks on paper because Random Forest used substantially more model complexity and still lost to a regularised linear model.

MODEL	MAE	RMSE	R^2	VERDICT
Weekly Ridge	0.011809	0.029473	0.993952	Best overall across all metrics
RF Deep	0.024842	0.065241	0.970365	Best Random Forest, still well behind Ridge
RF Default	0.025118	0.065962	0.969707	Marginal difference from deep variant
RF Simple	0.027664	0.068066	0.967743	Weakest configuration tested

Model	MAE	RMSE	R^2
Weekly Ridge	0.011809	0.029473	0.993952
RF Deep	0.024842	0.065241	0.970365
RF Default	0.025118	0.065962	0.969707
RF Simple	0.027664	0.068066	0.967743

Table 5.6. Weekly model comparison metrics

FIG 07 Weekly model comparison. Ridge delivered lower error and higher R^2 than every Random Forest variant. The actual numbers from the test set, not a cherry-picked run.

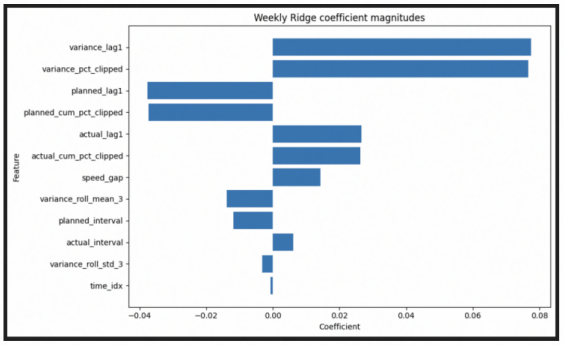


Figure 5.7. Weekly Ridge coefficient magnitudes

FIG 08 Weekly Ridge coefficient magnitudes. Variance lag and current variance carried the strongest signal. The model distributed weight across features rather than concentrating on one, which is exactly what regularisation was designed to produce.

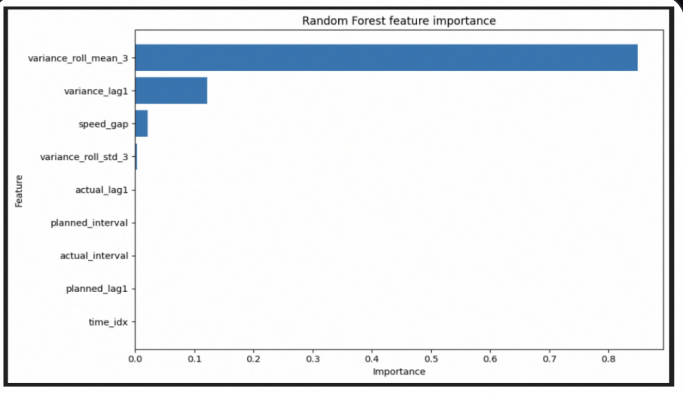


Figure 5.10. Random Forest feature importance

FIG 09 Random Forest feature importance. variance_roll_mean_3 dominates at over 0.8 importance, with almost nothing left for the remaining features. This concentration explains precisely why Random Forest underperformed despite its flexibility.

Appendix C. Package-Level prediction and diagnostic evidence

This appendix provides an illustrative package-level example from the weekly test set to show that the implemented forecasting pipeline produces predictions for individual packages rather than only aggregate summaries.

Row ID		Package	Date	Actual next-step variance	Ridge prediction	RF prediction	Naive prediction	Ridge abs. error	RF abs. error	Naive abs. error
0	2	Development and Upgrading Of Roads and Infrast...	2025-12-23	-0.256625	-0.252976	-0.260681	-0.243006	0.003649	0.004056	0.013619
1	2	Development and Upgrading Of Roads and Infrast...	2025-12-30	-0.262643	-0.262778	-0.266119	-0.248428	0.000135	0.003476	0.014215
2	2	Development and Upgrading Of Roads and Infrast...	2026-01-06	-0.268715	-0.267191	-0.268585	-0.256625	0.001524	0.000130	0.012090
3	2	Development and Upgrading Of Roads and Infrast...	2026-01-13	-0.274769	-0.273640	-0.271634	-0.262643	0.001129	0.003135	0.012126
4	2	Development and Upgrading Of Roads and Infrast...	2026-01-20	-0.283815	-0.279210	-0.273364	-0.268715	0.004606	0.010451	0.015100
5	2	Development and Upgrading Of Roads and Infrast...	2026-01-27	-0.290952	-0.289552	-0.295099	-0.274769	0.001400	0.004147	0.016183
6	2	Development and Upgrading Of Roads and Infrast...	2026-02-03	-0.297756	-0.295694	-0.299373	-0.283815	0.002063	0.001617	0.013941
7	2	Development and Upgrading Of Roads and Infrast...	2026-02-10	-0.302117	-0.301877	-0.300467	-0.290952	0.000240	0.001650	0.011165
8	2	Development and Upgrading Of Roads and Infrast...	2026-02-17	-0.304205	-0.303904	-0.300791	-0.297756	0.000301	0.003414	0.006449
9	2	Development and Upgrading Of Roads and Infrast...	2026-02-24	-0.305226	-0.303832	-0.301631	-0.302117	0.001394	0.003595	0.003109

Figure C.1: Detailed weekly package-level prediction comparison for the selected package "Development and Upgrading Of Roads and Infrastructure In The Mafraq Industrial City

FIG 10 Package-level prediction output for a real work package from the weekly test set. Ridge absolute error stays consistently below 0.005 across every row. The pipeline produces forecasts at the level where project teams actually make decisions.

WHY INTERPRETABILITY MATTERED AS MUCH AS ACCURACY

The Ridge coefficient chart makes the model readable. I could explain exactly which signals it relied on and why they made sense given the feature engineering logic. A model nobody trusts gets ignored regardless of how well it scores on a test set. Being able to show the reasoning behind the prediction is what makes it usable in a real project environment.

06 WHAT THIS DEMONSTRATES

This project sits beyond dashboards.

The skills here are about taking messy operational data with no clean schema, understanding what analytical question it can actually support, building a reproducible pipeline to reshape it, engineering features that reflect domain knowledge rather than just available columns, comparing models honestly using the correct evaluation method, and explaining results in a way that connects back to the business problem rather than hiding behind metrics.

01 **End-to-end Python pipeline**

Pandas, NumPy and Scikit-learn across ingestion, cleaning, transformation, feature engineering, model training, evaluation and output. Every stage reproducible from the raw workbook.

02 **Feature engineering from domain knowledge**

Five distinct feature families designed around how construction progress actually behaves over time, not just what columns were available in the source.

03 **Rigorous model comparison**

Chronological train-test splits, multiple configurations tested, results interpreted through the lens of the data structure rather than treated as a simple league table.

04

Package-level outputs, not portfolio averages

Forecasts generated at individual work package level because project controls teams manage through packages. An aggregate average is not where the action happens.

05

Algorithm choice driven by data understanding

The decision to use Ridge over more complex alternatives came from understanding the feature correlation structure, not defaulting to the most impressive-sounding method. That judgement is what separates analytical thinking from analytical pattern-matching.

Where this goes in practice

The most useful next version of this system connects package-level variance forecasts to critical path data. A negative forecast on a critical activity with no float is a fundamentally different risk from the same prediction on a non-critical package. Connecting model output to scheduling logic would allow early warning signals to be prioritised automatically rather than reviewed uniformly across all packages.

For any organisation running progress reporting today, this approach does not require new data infrastructure. It requires reshaping what already exists into the right structure and applying the right forecasting logic on top. The data is almost always already there.